Suavizado del perfil de tracks GPS mediante la media móvil gaussiana de Arnaud Legoux (ALMA)

Javier Urrutia
Ldo. en Ciencias Físicas y Arquitecto de Sistemas
javi@mendikat.net
Mendikat — `https://www.mendikat.net`

Octubre de 2026

Resumen

El desnivel positivo y negativo acumulado de un recorrido es uno de los datos más consultados por montañeros y senderistas, y también uno de los más difíciles de calcular con fiabilidad a partir de un track GPS: el ruido de la altitud registrada se acumula punto a punto y puede multiplicar el desnivel real. Este documento describe el método de análisis de elevaciones que Mendikat emplea desde la versión 18.6 (octubre de 2026): el perfil de elevaciones se suaviza con una media móvil gaussiana centrada —la media móvil de Arnaud Legoux (ALMA)— cuya ventana se fija en metros recorridos y no en número de puntos. Se repasan los métodos anteriores (umbral fijo e histéresis con umbral adaptativo), la familia de medias móviles de la que procede la ALMA y los resultados del estudio realizado sobre dos muestras de 1000 tracks, que justifican una ventana de 100 m.

Índice

1. Introducción **3**

2. El problema: el análisis de elevaciones de un track GPS **3**
2.1. Desnivel positivo y negativo 3
2.2. El efecto del ruido 3

3. Suavizado mediante un umbral de desnivel fijo **4**

4. Análisis de elevaciones por histéresis con umbral adaptativo **4**

5. El concepto de media móvil **4**

6. Medias móviles en el análisis técnico de los mercados financieros **5**

7. La media móvil simple (SMA) **5**

8. La media móvil ponderada (WMA) **6**

9. La media móvil exponencial (EMA) **6**

10. La media móvil gaussiana o de Arnaud Legoux (ALMA) **6**

11. Análisis de elevaciones por media móvil gaussiana **7**

12. Por qué usar la media móvil gaussiana en el análisis de elevaciones **8**

13. Por qué la ventana se fija en metros y no en número de puntos **8**

14. Conclusiones **9**

A. Ejemplo de implementación en Python **11**

B. Implementación en Java **14**

C. Implementación en PHP **17**

D. Implementación en Rust **20**

1. Introducción

Un *track* GPS es una secuencia ordenada de puntos $P_i = (\varphi_i, \lambda_i, h_i)$, $i = 1, \ldots, n$, con latitud, longitud y altitud, registrada por un receptor a intervalos regulares de tiempo o de distancia. A partir de él se obtienen las magnitudes que describen un recorrido: distancia, altitudes máxima y mínima, duración y, en particular, el *desnivel positivo* (metros ascendidos) y el *desnivel negativo* (metros descendidos) acumulados.

La altitud es la coordenada que peor resuelve un receptor GNSS. La geometría de los satélites, siempre por encima del observador, hace que el error vertical sea típicamente entre 1,5 y 2 veces el horizontal [7], y a ello se suman los efectos del relieve, la vegetación y el multitrayecto, muy frecuentes en montaña. Los aparatos con altímetro barométrico reducen el ruido, pero introducen derivas por los cambios de presión atmosférica. En ambos casos, el perfil registrado es la suma del perfil real y de una componente de ruido de alta frecuencia.

Mendikat calcula el desnivel de los tracks que publican sus usuarios. Como cada track procede de un aparato y de unas condiciones distintas, el método debe ser robusto frente a la frecuencia de registro, la magnitud del ruido y la longitud del recorrido, sin necesidad de parámetros ajustados a mano para cada caso. Este documento describe la evolución de ese método y justifica la solución adoptada: un suavizado gaussiano con una ventana de anchura fija en metros.

2. El problema: el análisis de elevaciones de un track GPS

2.1. Desnivel positivo y negativo

Dado un perfil de elevaciones $h_1, \ldots, h_n$, el desnivel positivo y el negativo acumulados se definen como

$$D^+ = \sum_{i=2}^{n} \max\left(h_i - h_{i-1},\, 0\right), \qquad D^- = \sum_{i=2}^{n} \max\left(h_{i-1} - h_i,\, 0\right). \tag{1}$$

Se cumple que $D^+ - D^- = h_n - h_1$, de modo que en un circuito, cuyo punto final coincide con el inicial, ambos desniveles deben ser iguales.

2.2. El efecto del ruido

Supongamos que la altitud registrada es $h_i = z_i + \varepsilon_i$, donde z_i es la altitud real y ε_i un ruido de media nula. En un tramo llano (z_i constante), cada diferencia $h_i - h_{i-1}$ es pura diferencia de ruido y, por la definición (1), la mitad de ellas se suman a D^+ y la otra mitad a D^-. Si el ruido es gaussiano, independiente y de desviación típica σ_ε, la diferencia entre dos lecturas tiene desviación $\sqrt{2}\,\sigma_\varepsilon$ y su parte positiva tiene esperanza

$$\mathbb{E}\left[\max(\varepsilon_i - \varepsilon_{i-1}, 0)\right] = \frac{\sqrt{2}\,\sigma_\varepsilon}{\sqrt{2\pi}} = \frac{\sigma_\varepsilon}{\sqrt{\pi}} \approx 0{,}56\,\sigma_\varepsilon. \tag{2}$$

Es decir, el ruido añade aproximadamente $0{,}56\,\sigma_\varepsilon$ metros de desnivel *por punto*. Con un ruido de 3 m y un punto cada 10 m, un recorrido de 10 km acumula unos 1700 m de desnivel ficticio, varias veces el desnivel real de cualquier excursión de esa longitud. El error crece linealmente con el número de puntos: **cuantos más puntos registra el aparato, más desnivel ficticio se acumula**.

El objetivo de cualquier método de análisis es, por tanto, separar el perfil real z_i del ruido ε_i antes de aplicar (1), eliminando las oscilaciones de alta frecuencia sin aplanar las cimas, los collados y los cambios de pendiente reales.

3. Suavizado mediante un umbral de desnivel fijo

El método más sencillo consiste en ignorar las variaciones de altitud menores que un umbral u (por ejemplo, $u = 10$ m). El perfil se recorre manteniendo una tendencia (subida o bajada) y un extremo candidato; un cambio de tendencia sólo se acepta cuando la altitud se aleja del extremo candidato más de u metros. El resultado es un perfil simplificado formado únicamente por las cimas y los valles cuya prominencia relativa supera el umbral, sobre el que se calcula el desnivel con (1).

Este filtro es un *filtro de histéresis*: una oscilación de amplitud menor que u nunca llega a registrarse como subida o bajada. Es eficaz y fácil de entender, pero tiene un inconveniente fundamental: **ningún umbral fijo sirve para todos los tracks**.

- Un umbral pequeño deja pasar el ruido de los aparatos con poca precisión vertical.
- Un umbral grande elimina desniveles reales: las lomas, los collados y los resaltes de menos de u metros desaparecen del perfil, lo que penaliza los recorridos rompepiernas.

4. Análisis de elevaciones por histéresis con umbral adaptativo

Para resolver la dependencia del umbral, Mendikat empleó hasta la versión 18.6 una histéresis con *umbral adaptativo*: para cada track se prueban de forma iterativa umbrales crecientes, desde 2 hasta 36 m, y el proceso se detiene cuando el desnivel calculado con dos umbrales consecutivos difiere en menos de 3 m. La idea es que, una vez eliminado el ruido, aumentar el umbral deja de modificar el resultado: el desnivel se *estabiliza*.

El estudio realizado con este método permitió extraer varias conclusiones:

1. El umbral de estabilización varía mucho de un track a otro, entre 3 y 23 m: ningún valor fijo sirve igual para todos.
2. Aproximadamente el 20 % de los tracks se estabiliza en un umbral de 3 m, sin que llegue a filtrarse nada.
3. En torno al 75 % de los tracks tiene un umbral de hasta 10 m, valor razonable en la mayoría de los casos.
4. Los tracks cortos y con poco desnivel suelen tener mucho ruido.
5. Cuantos más puntos por kilómetro registra el aparato, más ruido.

Que el umbral se adapte a cada track está justificado, porque el ruido cambia mucho de una grabación a otra. Sin embargo, el punto débil del método es su **regla de parada**: se detiene en el primer escalón en el que el desnivel deja de cambiar, que no tiene por qué coincidir con el nivel en el que termina el ruido. Además, el resultado es discontinuo: un pequeño cambio en el track puede desplazar el umbral elegido y, con él, el desnivel calculado en decenas de metros. Estas limitaciones motivaron la búsqueda de un método basado en un filtro continuo.

5. El concepto de media móvil

Una *media móvil* de orden k sustituye cada valor de una serie por un promedio de los k valores de una ventana que se desplaza a lo largo de ella. En su forma general, la media móvil ponderada de la serie $x_1, \ldots, x_n$ es

$$\bar{x}_i = \frac{\sum_{j=0}^{k-1} w_j \, x_{i+j}}{\sum_{j=0}^{k-1} w_j}, \qquad i = 1, \ldots, n-k+1, \tag{3}$$

donde los pesos $w_j \geq 0$ definen el tipo de media. Desde el punto de vista del tratamiento de señales, (3) es la convolución de la serie con un *núcleo* discreto de k coeficientes normalizados: un *filtro paso bajo* de respuesta finita (FIR) que atenúa las variaciones rápidas (el ruido) y conserva las lentas (la tendencia).

Dos propiedades caracterizan una media móvil:

- **El suavizado**, que depende de la anchura de la ventana y de la forma de los pesos: cuanto más ancha es la ventana, más ruido se elimina, pero también más detalle real.
- **El retardo**, que depende de la posición del centro de masas de los pesos dentro de la ventana. Si los pesos están centrados, el filtro no desplaza la señal; si se concentran en un extremo, la señal filtrada se adelanta o retrasa respecto a la original.

Nótese que (3) devuelve $n - k + 1$ valores: los $k - 1$ extremos de la serie quedan sin un valor filtrado completo, cuestión que habrá que resolver al aplicarla a un perfil de elevaciones.

6. Medias móviles en el análisis técnico de los mercados financieros

Las medias móviles son una de las herramientas más antiguas y extendidas del *análisis técnico* [3], el estudio de la evolución de los precios de un activo financiero a partir de su propia serie histórica. Aplicadas a la serie de precios de cierre, permiten identificar la tendencia subyacente filtrando las fluctuaciones diarias; los cruces del precio con su media, o entre una media rápida (ventana corta) y una lenta (ventana larga), se interpretan como señales de cambio de tendencia.

En este contexto la serie se conoce sólo hasta el instante presente, de modo que los filtros son necesariamente *causales*: únicamente pueden usar valores pasados. Esto convierte el retardo en el problema central: una media que suaviza mucho reacciona tarde a los cambios del precio. La historia de las medias móviles financieras —simple, ponderada, exponencial y, finalmente, la de Arnaud Legoux— es en buena medida la historia de la búsqueda del mejor compromiso entre suavizado y retardo.

Un perfil de elevaciones, en cambio, se conoce completo antes de analizarlo: no hay un *presente* y no hace falta que el filtro sea causal. Esta diferencia será decisiva al elegir los parámetros de la ALMA (sección 11).

7. La media móvil simple (SMA)

La media móvil simple (*Simple Moving Average*) asigna el mismo peso a todos los valores de la ventana:

$$\mathrm{SMA}_i = \frac{1}{k} \sum_{j=0}^{k-1} x_{i+j}. \tag{4}$$

Es la media más fácil de calcular e interpretar, pero presenta dos inconvenientes. Primero, su núcleo rectangular tiene una respuesta en frecuencia de tipo sinc [5], con lóbulos secundarios que dejan pasar parte del ruido de alta frecuencia e incluso invierten el signo de algunas componentes. Segundo, todos los valores de la ventana pesan igual, de modo que un valor atípico influye con la misma intensidad durante k posiciones y desaparece bruscamente al salir de la ventana, produciendo escalones en la serie suavizada. Usada de forma causal, su retardo es de $(k - 1)/2$ posiciones.

8. La media móvil ponderada (WMA)

La media móvil ponderada (*Weighted Moving Average*) asigna pesos que decrecen linealmente desde el valor más reciente ($w = k$) hasta el más antiguo ($w = 1$):

$$\mathrm{WMA}_i = \frac{\sum_{j=0}^{k-1}(k-j)\,x_{i+j}}{\sum_{j=0}^{k-1}(k-j)} = \frac{2}{k(k+1)}\sum_{j=0}^{k-1}(k-j)\,x_{i+j}, \tag{5}$$

donde x_i es el valor más reciente de la ventana. Al dar más importancia a los valores recientes, la WMA reduce el retardo respecto a la SMA y la salida de un valor de la ventana es gradual. A cambio, suaviza menos con la misma ventana y su núcleo, asimétrico, sigue teniendo un corte abrupto en el extremo más reciente.

9. La media móvil exponencial (EMA)

La media móvil exponencial (*Exponential Moving Average*) se define de forma recursiva:

$$\mathrm{EMA}_i = \alpha\,x_i + (1-\alpha)\,\mathrm{EMA}_{i+1}, \qquad \alpha = \frac{2}{k+1}, \tag{6}$$

donde EMA_{i+1} es el valor de la media en el instante anterior. Desarrollando la recurrencia se obtiene un núcleo de pesos $w_j = \alpha(1-\alpha)^j$ que decrecen geométricamente y nunca llegan a anularse: cada valor del pasado influye, cada vez menos, en todos los valores posteriores [4]. La EMA es barata de calcular, reacciona rápido a los cambios y no presenta los escalones de la SMA, por lo que es la media más usada en el análisis técnico. Su núcleo, sin embargo, es muy asimétrico: es un filtro con memoria y retardo intrínsecos.

10. La media móvil gaussiana o de Arnaud Legoux (ALMA)

La media móvil de Arnaud Legoux (*Arnaud Legoux Moving Average*, ALMA) fue presentada en 2009 por Arnaud Legoux y Dimitrios Kouzis-Loukas [2, 1] con el objetivo de reducir el retardo de las medias clásicas sin renunciar al suavizado. Su idea es emplear como núcleo una curva de Gauss cuyo centro puede desplazarse dentro de la ventana.

Para una ventana de k valores, indexados por su posición $j = 0, \ldots, k-1$ (de la más antigua a la más reciente), los pesos son

$$w_j = \exp\left(-\frac{(j-m)^2}{2s^2}\right), \qquad m = \mathit{offset}\cdot(k-1), \qquad s = \frac{k}{\sigma}, \tag{7}$$

y la media se obtiene con la expresión general (3). Los tres parámetros del filtro son:

- **La ventana** k: el número de valores que se promedian.
- **El desplazamiento** $\mathit{offset} \in [0, 1]$: la posición del centro de la gaussiana dentro de la ventana. Con $\mathit{offset} = 0{,}5$ la gaussiana está centrada y el filtro es simétrico; con valores próximos a 1 los pesos se concentran en los valores más recientes y el retardo se reduce. En el análisis técnico el valor recomendado es 0,85.
- **El factor de anchura** σ: la ventana contiene σ desviaciones típicas de la gaussiana. El valor habitual, $\sigma = 6$, inspirado en el método *Seis Sigma*, hace que la ventana abarque ± 3 desviaciones típicas alrededor del centro.

La figura 1 compara los pesos normalizados de la SMA, la WMA y la ALMA centrada para una ventana de $k = 9$ valores.

La gaussiana tiene propiedades que la hacen especialmente adecuada como núcleo de suavizado [6]:

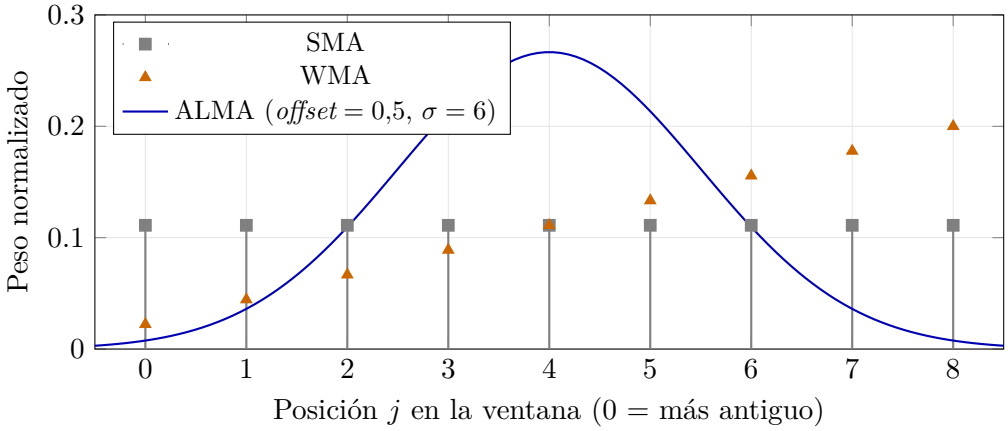


Figura 1: Pesos normalizados de la SMA, la WMA y la ALMA centrada para una ventana de 9 valores. La ALMA concentra el peso en el centro de la ventana y lo reduce suavemente hacia los extremos.

- Su transformada de Fourier es otra gaussiana: la respuesta en frecuencia del filtro decrece de forma monótona, sin los lóbulos secundarios de la SMA. Atenúa el ruido de alta frecuencia sin introducir oscilaciones espurias.
- Es suave y estrictamente positiva: cada valor entra y sale de la ventana con un peso muy pequeño (con $\sigma = 6$, el peso en los extremos es $e^{-9/2} \approx 0{,}011$ veces el central), de modo que no aparecen escalones al desplazar la ventana.
- Es el núcleo que minimiza el producto de la anchura en el espacio y en la frecuencia: para un mismo grado de suavizado, es el que menos desdibuja los detalles.

11. Análisis de elevaciones por media móvil gaussiana

El analizador de elevaciones de Mendikat desde la versión 18.6+ aplica la ALMA al perfil de elevaciones de cada segmento de track en cinco pasos.

1. Serie de altitudes. Se extraen las altitudes de los puntos del segmento, descartando los puntos sin altitud. Si el segmento no tiene ninguna, el perfil se reduce a un único valor nulo.

2. Ventana en puntos a partir de una ventana en metros. La anchura de la ventana se fija en $W = 100$ m de recorrido. Para convertirla en un número de puntos se calcula la separación media entre puntos consecutivos *en movimiento*:

$$\bar{d} = \frac{1}{|M|} \sum_{i \in M} d(P_{i-1}, P_i), \qquad M = \{\, i : d(P_{i-1}, P_i) > d_{\text{mín}} \,\}, \tag{8}$$

donde d es la distancia geodésica entre dos puntos y $d_{\text{mín}} = 1$ m. Los tramos sin desplazamiento (paradas) se excluyen porque reducirían la separación media y agrandarían la ventana real en los tramos en movimiento. El número de puntos de la ventana es

$$k = \text{mín}\left(\text{máx}\left(\text{round}\left(\frac{W}{\bar{d}}\right), 1\right), n\right). \tag{9}$$

Como $\bar{d} > d_{\text{mín}}$, la ventana tiene como mucho $W/d_{\text{mín}} = 100$ puntos. Si los puntos están más separados que la propia ventana, $k = 1$ y el perfil no se suaviza; si no hay ningún tramo en movimiento, se aplica la ventana máxima, ya que la altitud sólo puede variar por el ruido.

3. Prolongación de los extremos. La media móvil devuelve $n - k + 1$ valores. Para no perder el desnivel del principio y del final del recorrido, el perfil se prolonga repitiendo el primer valor $\lfloor (k-1)/2 \rfloor$ veces al principio y el último valor $k - 1 - \lfloor (k-1)/2 \rfloor$ veces al final, de modo que el perfil suavizado conserva los n valores del original.

4. Suavizado gaussiano centrado. Se aplica la ALMA (7) con *offset* = 0,5 y $\sigma = 6$. El desplazamiento centrado es la diferencia clave respecto al uso financiero: en un perfil de elevaciones no hay valores *más recientes* que deban pesar más, y una gaussiana centrada es un filtro simétrico, de fase nula, que no desplaza las cimas ni los collados a lo largo del recorrido.

Con estos parámetros, la desviación típica del núcleo es $s = k/6$ puntos, es decir, aproximadamente

$$s \cdot \bar{d} \approx \frac{W}{6} \approx 16{,}7 \text{ m} \tag{10}$$

de recorrido, *con independencia de la densidad de puntos del track.*

5. Desnivel, altitudes extremas y circuitos. El desnivel positivo y negativo se calcula con (1) sobre el perfil suavizado. Las altitudes máxima y mínima, en cambio, se toman del perfil original, ya que el suavizado rebaja las cimas y eleva los valles. Por último, si el punto final del track está a menos de 100 m del inicial, el recorrido se considera un circuito y se iguala el desnivel negativo al positivo.

12. Por qué usar la media móvil gaussiana en el análisis de elevaciones

- **Es un filtro continuo.** El resultado varía de forma continua con los datos: no hay umbrales ni reglas de parada, y pequeñas diferencias entre dos tracks del mismo recorrido producen pequeñas diferencias en el desnivel.
- **Actúa sobre la causa del error.** Como se vio en la sección 2, el desnivel ficticio procede del ruido de alta frecuencia, que es precisamente lo que un filtro paso bajo gaussiano atenúa, sin lóbulos secundarios que lo dejen pasar.
- **No introduce desplazamientos.** Al ser un filtro centrado (fase nula), las cimas, los collados y los cambios de pendiente permanecen en su posición. Esto sólo es posible porque el perfil se conoce completo: la ventaja que la ALMA busca en el análisis técnico (reducir el retardo desplazando la gaussiana) se convierte aquí en la eliminación total del retardo.
- **Respeta la forma del relieve.** El desnivel de una subida larga y sostenida es prácticamente insensible al suavizado; sólo se ven afectadas las variaciones de escala comparable a la ventana, es decir, las de unas pocas decenas de metros de recorrido.
- **Es sencilla y eficiente.** El coste es $O(n\,k)$ con $k \leq 100$, y el método depende de un único parámetro con significado físico claro: la anchura de la ventana en metros.

Como contrapartida, el suavizado redondea ligeramente las cimas y los collados agudos, por lo que en tracks con poco ruido el desnivel obtenido es algo inferior al de la histéresis (sección 14).

13. Por qué la ventana se fija en metros y no en número de puntos

En el análisis técnico la ventana se expresa en número de periodos (por ejemplo, 9 sesiones), porque los precios se registran a intervalos regulares. Los tracks GPS no cumplen esa condición:

según el aparato y su configuración, pueden registrar un punto cada segundo, cada pocos metros o cada varios minutos. Una ventana de 9 puntos abarca así unos 45 m de recorrido en un track con un punto cada 5 m, y más de 450 m en uno con menos de 20 puntos por kilómetro.

Una ventana fija en número de puntos aplica, por tanto, un suavizado muy distinto a dos tracks del mismo recorrido. En los tracks densos apenas filtra el ruido, que es precisamente mayor cuantos más puntos se registran; en los tracks poco densos borra cimas y collados reales. El estudio realizado lo confirma: en tracks con menos de 20 puntos/km, una ventana de 9 puntos elimina un 20 % del desnivel real.

Fijar la ventana en metros hace que el filtro actúe siempre sobre la misma escala física del relieve: la desviación típica del núcleo es de unos 16,7 m de recorrido tanto si el track tiene un punto cada 2 m como si tiene uno cada 20 m. El resultado deja de depender de la frecuencia de registro del aparato.

14. Conclusiones

El nuevo método se evaluó en octubre de 2026 sobre dos muestras: los 1000 últimos tracks publicados en Mendikat y 1000 tracks distintos elegidos al azar con más de 500 puntos cada uno, comparando su desnivel con el obtenido por el método anterior de histéresis con umbral adaptativo. Las conclusiones son:

1. **La ventana debe fijarse en metros.** Con una ventana fija en número de puntos, el resultado depende de la densidad de puntos: en tracks con menos de 20 puntos/km, una ventana de 9 puntos elimina un 20 % del desnivel real.
2. **Una ventana de 100 m es la que más se aproxima a la histéresis**, con una diferencia mediana de +0,1 % y el 88 % de los tracks dentro de ±10 %, con independencia de la densidad de puntos.
3. **Las diferencias son sistemáticas y explicables.** En tracks limpios, la media gaussiana obtiene algo menos de desnivel que la histéresis (−1,5 %), porque redondea cimas y collados; en tracks ruidosos, en los que la histéresis aplica umbrales grandes que eliminan desniveles reales, obtiene algo más (+4 %).

La media móvil gaussiana centrada con una ventana de 100 m reproduce, por tanto, los resultados de la histéresis en la gran mayoría de los tracks, corrige sus errores en los más ruidosos y sustituye un procedimiento iterativo con regla de parada por un filtro continuo, simétrico y con un único parámetro de significado físico claro. Por estas razones sustituye al análisis por histéresis en Mendikat desde la versión 18.6 (octubre de 2026).

Referencias

[1] TradingView. *Media móvil de Arnaud Legoux (ALMA).* `https://es.tradingview.com/support/solutions/43000594683/`

[2] A. Legoux y D. Kouzis-Loukas. *ALMA: Arnaud Legoux Moving Average.* 2009.

[3] J. J. Murphy. *Technical Analysis of the Financial Markets.* New York Institute of Finance, 1999.

[4] J. S. Hunter. The exponentially weighted moving average. *Journal of Quality Technology*, 18(4):203–210, 1986.

[5] S. W. Smith. *The Scientist and Engineer's Guide to Digital Signal Processing*, cap. 15: *Moving Average Filters.* California Technical Publishing, 1997.

[6] R. A. Haddad y A. N. Akansu. A class of fast Gaussian binomial filters for speech and image processing. *IEEE Transactions on Signal Processing*, 39(3):723–727, 1991.

[7] E. D. Kaplan y C. J. Hegarty (eds.). *Understanding GPS/GNSS: Principles and Applications*, 3.ª ed. Artech House, 2017.

[8] R. W. Sinnott. Virtues of the Haversine. *Sky and Telescope*, 68(2):159, 1984.

[9] Mendikat. Código fuente: `lib/GPS/Elevation/GaussianElevationAnalyzer.php` y `lib/Math/Statistics/MovingAverage.php`, versión 18.6. `https://www.mendikat.net`

A. Ejemplo de implementación en Python

El siguiente programa reproduce el algoritmo descrito en la sección 11. Recibe una lista de puntos `(lat, lon, ele)` y devuelve el desnivel positivo y negativo y las altitudes mínima y máxima. La distancia entre puntos se calcula con la fórmula del haversine [8]. Como la gaussiana está centrada, el orden de los valores dentro de la ventana es indiferente y no es necesario invertir la serie, como hace la implementación financiera.

Para que el ejemplo sea reproducible en cualquier lenguaje, el ruido del track sintético no se obtiene del generador aleatorio de la biblioteca estándar, sino de un generador congruencial de Park y Miller ($x_{i+1} = 48271\,x_i \bmod (2^{31} - 1)$) y de la transformación de Box-Muller. Las implementaciones de los apéndices B, C y D usan el mismo generador y la misma semilla, y producen exactamente el mismo resultado.

```python
"""
Análisis de elevaciones de un track GPS mediante la media móvil gaussiana (ALMA)
con una ventana de suavizado fijada en metros.
"""

import math

WINDOW = 100.0      # Anchura de la ventana de suavizado (m)
OFFSET = 0.5        # Centro de la gaussiana dentro de la ventana (centrada)
SIGMA = 6.0         # Factor de anchura de la gaussiana
MIN_STEP = 1.0      # Separación mínima (m) para considerar que hay desplazamiento
CIRCUIT = 100.0     # Distancia máxima (m) entre inicio y fin de un circuito
R = 6371008.8       # Radio medio terrestre (m)


def distance(p, q):
    """Distancia (m) entre dos puntos (lat, lon) por la fórmula del haversine."""
    lat1, lon1, lat2, lon2 = map(math.radians, (p[0], p[1], q[0], q[1]))
    h = (math.sin((lat2 - lat1) / 2) ** 2
         + math.cos(lat1) * math.cos(lat2) * math.sin((lon2 - lon1) / 2) ** 2)
    return 2 * R * math.asin(math.sqrt(h))


def window_points(points, n):
    """Número de puntos k de la ventana a partir de la separación media en movimiento.
    """
    steps = [d for d in (distance(points[i - 1], points[i])
                         for i in range(1, len(points))) if d > MIN_STEP]
    if not steps:
        return min(int(WINDOW / MIN_STEP), n)
    spacing = sum(steps) / len(steps)
    return min(max(math.floor(WINDOW / spacing + 0.5), 1), n)


def alma_weights(k, offset=OFFSET, sigma=SIGMA):
    """Pesos gaussianos de la ALMA para una ventana de k puntos."""
    m = offset * (k - 1)
    s = k / sigma
    return [math.exp(-((j - m) ** 2) / (2 * s ** 2)) for j in range(k)]


def smooth(elevation, k):
    """Perfil suavizado con n valores: se prolongan los extremos antes de filtrar."""
    before = (k - 1) // 2
```

```
    after = k - 1 - before
    padded = [elevation[0]] * before + elevation + [elevation[-1]] * after
    w = alma_weights(k)
    total = sum(w)
    return [sum(padded[i + j] * w[j] for j in range(k)) / total
            for i in range(len(elevation))]


def gain_and_loss(profile):
    """Desnivel positivo y negativo acumulados de un perfil."""
    gain = sum(max(b - a, 0.0) for a, b in zip(profile, profile[1:]))
    loss = sum(max(a - b, 0.0) for a, b in zip(profile, profile[1:]))
    return gain, loss


def analyze(points):
    """
    points: lista de tuplas (lat, lon, ele); ele puede ser None.
    Devuelve el desnivel positivo y negativo y las altitudes mínima y máxima.
    """
    elevation = [float(p[2]) for p in points if p[2] is not None] or [0.0]
    k = window_points(points, len(elevation))
    gain, loss = gain_and_loss(smooth(elevation, k))
    if distance(points[0], points[-1]) < CIRCUIT:   # Circuito: D+ = D-
        loss = gain
    return {"k": k, "gain": gain, "loss": loss,
            "min": min(elevation), "max": max(elevation)}


class Noise:
    """Ruido gaussiano reproducible: generador de Park y Miller y Box-Muller."""

    def __init__(self, seed):
        self.state = seed

    def uniform(self):
        self.state = self.state * 48271 % 2147483647
        return self.state / 2147483647

    def gauss(self, sigma):
        u1, u2 = self.uniform(), self.uniform()
        return sigma * math.sqrt(-2 * math.log(u1)) * math.cos(2 * math.pi * u2)


if __name__ == "__main__":
    # Track sintético: subida de 500 m y bajada en 10 km, un punto cada 10 m,
    # con ruido gaussiano de 3 m en la altitud
    noise = Noise(1)
    n = 1001
    track = []
    for i in range(n):
        x = i / (n - 1)
        ele = 1000 + 500 * math.sin(math.pi * x) + noise.gauss(3)
        track.append((43.0 + 0.09 * x, -2.0, ele))

    raw_gain, _ = gain_and_loss([p[2] for p in track])
    result = analyze(track)
    print(f"Desnivel sin suavizar: {raw_gain:.0f} m")
```

```
    print(f"ALMA (k = {result['k']}): D+ {result['gain']:.0f} m, "
          f"D- {result['loss']:.0f} m")
    print(f"Altitud: min {result['min']:.0f} m, max {result['max']:.0f} m")
```

Al ejecutarlo sobre el track sintético del ejemplo —un recorrido de 10 km con un punto cada 10 m, un desnivel real de 500 m de subida y 500 m de bajada y un ruido gaussiano de 3 m en la altitud— se obtiene:

```
Desnivel sin suavizar: 1735 m
ALMA (k = 10): D+ 527 m, D- 533 m
Altitud: min 997 m, max 1507 m
```

Sin suavizar, el ruido multiplica por 3,5 el desnivel real, en línea con la estimación de $0{,}56\,\sigma_\varepsilon$ metros por punto de la sección 2 ($1000 \times 0{,}56 \times 3 \approx 1700$ m). Con la media gaussiana sobre una ventana de 100 m (10 puntos), el error se reduce a en torno a un 6 %. Las altitudes mínima y máxima, tomadas del perfil sin suavizar, incluyen el ruido de las lecturas extremas.

B. Implementación en Java

Implementación equivalente en Java (versión 17 o posterior, por el uso de `record`) que produce la misma salida que el programa del apéndice A. Se compila y ejecuta con:

```
javac AlmaElevation.java && java AlmaElevation
```

```java
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;

/**
 * Análisis de elevaciones de un track GPS mediante la media móvil gaussiana (ALMA)
 * con una ventana de suavizado fijada en metros.
 */
public final class AlmaElevation {

    static final double WINDOW = 100.0;     // Anchura de la ventana de suavizado (m)
    static final double OFFSET = 0.5;       // Centro de la gaussiana (centrada)
    static final double SIGMA = 6.0;        // Factor de anchura de la gaussiana
    static final double MIN_STEP = 1.0;     // Separación mínima (m) en movimiento
    static final double CIRCUIT = 100.0;    // Distancia máxima (m) inicio-fin de un
    circuito
    static final double R = 6371008.8;      // Radio medio terrestre (m)

    /** Un punto del track; ele es null si no hay altitud. */
    record Point(double lat, double lon, Double ele) {}

    /** El resultado del análisis. */
    record Analysis(int k, double gain, double loss, double min, double max) {}

    /** Distancia (m) entre dos puntos por la fórmula del haversine. */
    static double distance(Point p, Point q) {
        double lat1 = Math.toRadians(p.lat()), lat2 = Math.toRadians(q.lat());
        double dLat = lat2 - lat1, dLon = Math.toRadians(q.lon() - p.lon());
        double h = Math.pow(Math.sin(dLat / 2), 2)
                 + Math.cos(lat1) * Math.cos(lat2) * Math.pow(Math.sin(dLon / 2), 2);
        return 2 * R * Math.asin(Math.sqrt(h));
    }

    /** Número de puntos k de la ventana a partir de la separación media en movimiento.
     */
    static int windowPoints(List<Point> points, int n) {
        double length = 0;
        int steps = 0;
        for (int i = 1; i < points.size(); i++) {
            double d = distance(points.get(i - 1), points.get(i));
            if (d > MIN_STEP) {
                length += d;
                steps++;
            }
        }
        if (steps == 0) {
            return Math.min((int) (WINDOW / MIN_STEP), n);
        }
        double spacing = length / steps;
        return (int) Math.min(Math.max(Math.round(WINDOW / spacing), 1), n);
    }

```

```
    /** Pesos gaussianos de la ALMA para una ventana de k puntos. */
    static double[] almaWeights(int k) {
        double m = OFFSET * (k - 1);
        double s = k / SIGMA;
        double[] w = new double[k];
        for (int j = 0; j < k; j++) {
            w[j] = Math.exp(-((j - m) * (j - m)) / (2 * s * s));
        }
        return w;
    }

    /** Perfil suavizado con n valores: se prolongan los extremos antes de filtrar. */
    static double[] smooth(double[] elevation, int k) {
        int n = elevation.length;
        int before = (k - 1) / 2;
        double[] padded = new double[n + k - 1];
        for (int i = 0; i < padded.length; i++) {
            int src = Math.min(Math.max(i - before, 0), n - 1);
            padded[i] = elevation[src];
        }
        double[] w = almaWeights(k);
        double total = 0;
        for (double wj : w) {
            total += wj;
        }
        double[] result = new double[n];
        for (int i = 0; i < n; i++) {
            double sum = 0;
            for (int j = 0; j < k; j++) {
                sum += padded[i + j] * w[j];
            }
            result[i] = sum / total;
        }
        return result;
    }

    /** Desnivel positivo y negativo acumulados de un perfil. */
    static double[] gainAndLoss(double[] profile) {
        double gain = 0, loss = 0;
        for (int i = 1; i < profile.length; i++) {
            double diff = profile[i] - profile[i - 1];
            if (diff >= 0) {
                gain += diff;
            } else {
                loss -= diff;
            }
        }
        return new double[] {gain, loss};
    }

    /** Analiza las elevaciones de un track. */
    static Analysis analyze(List<Point> points) {
        double[] elevation = points.stream()
                .filter(p -> p.ele() != null)
                .mapToDouble(Point::ele)
                .toArray();
        if (elevation.length == 0) {
            elevation = new double[] {0.0};
```

```
        }
        int k = windowPoints(points, elevation.length);
        double[] gl = gainAndLoss(smooth(elevation, k));
        double gain = gl[0], loss = gl[1];
        if (distance(points.get(0), points.get(points.size() - 1)) < CIRCUIT) {
            loss = gain;    // Circuito: D+ = D-
        }
        double min = Double.MAX_VALUE, max = -Double.MAX_VALUE;
        for (double e : elevation) {
            min = Math.min(min, e);
            max = Math.max(max, e);
        }
        return new Analysis(k, gain, loss, min, max);
    }

    /** Ruido gaussiano reproducible: generador de Park y Miller y Box-Muller. */
    static final class Noise {
        private long state;

        Noise(long seed) {
            state = seed;
        }

        double uniform() {
            state = state * 48271 % 2147483647;
            return state / 2147483647.0;
        }

        double gauss(double sigma) {
            double u1 = uniform(), u2 = uniform();
            return sigma * Math.sqrt(-2 * Math.log(u1)) * Math.cos(2 * Math.PI * u2);
        }
    }

    public static void main(String[] args) {
        // Track sintético: subida de 500 m y bajada en 10 km, un punto cada 10 m,
        // con ruido gaussiano de 3 m en la altitud
        Noise noise = new Noise(1);
        int n = 1001;
        List<Point> track = new ArrayList<>();
        double[] raw = new double[n];
        for (int i = 0; i < n; i++) {
            double x = (double) i / (n - 1);
            double ele = 1000 + 500 * Math.sin(Math.PI * x) + noise.gauss(3);
            track.add(new Point(43.0 + 0.09 * x, -2.0, ele));
            raw[i] = ele;
        }

        Analysis result = analyze(track);
        System.out.printf(Locale.ROOT, "Desnivel sin suavizar: %.0f m%n", gainAndLoss(
    raw)[0]);
        System.out.printf(Locale.ROOT, "ALMA (k = %d): D+ %.0f m, D- %.0f m%n",
                result.k(), result.gain(), result.loss());
        System.out.printf(Locale.ROOT, "Altitud: min %.0f m, max %.0f m%n", result.
    min(), result.max());
    }
}
```

C. Implementación en PHP

Implementación equivalente en PHP (versión 8.1 o posterior). Es una versión autocontenida y simplificada del analizador de Mendikat, sin dependencias del resto de la aplicación. Se ejecuta con `php alma_elevation.php` y produce la misma salida que el programa del apéndice A.

```php
<?php

/**
 * Análisis de elevaciones de un track GPS mediante la media móvil gaussiana (ALMA)
 * con una ventana de suavizado fijada en metros.
 */

declare(strict_types=1);

const WINDOW   = 100.0;     // Anchura de la ventana de suavizado (m)
const OFFSET   = 0.5;       // Centro de la gaussiana dentro de la ventana (centrada)
const SIGMA    = 6.0;       // Factor de anchura de la gaussiana
const MIN_STEP = 1.0;       // Separación mínima (m) para considerar que hay desplazamiento
const CIRCUIT  = 100.0;     // Distancia máxima (m) entre inicio y fin de un circuito
const R        = 6371008.8; // Radio medio terrestre (m)

/** Distancia (m) entre dos puntos [lat, lon] por la fórmula del haversine. */
function distance(array $p, array $q): float
{
    [$lat1, $lat2] = [deg2rad($p[0]), deg2rad($q[0])];
    $dLat = $lat2 - $lat1;
    $dLon = deg2rad($q[1] - $p[1]);
    $h = sin($dLat / 2) ** 2 + cos($lat1) * cos($lat2) * sin($dLon / 2) ** 2;

    return 2 * R * asin(sqrt($h));
}

/** Número de puntos k de la ventana a partir de la separación media en movimiento. */
function windowPoints(array $points, int $n): int
{
    $length = 0.0;
    $steps  = 0;
    for ($i = 1; $i < count($points); $i++) {
        $d = distance($points[$i - 1], $points[$i]);
        if ($d > MIN_STEP) {
            $length += $d;
            $steps++;
        }
    }
    if ($steps === 0) {
        return min((int) (WINDOW / MIN_STEP), $n);
    }
    $spacing = $length / $steps;

    return min(max((int) round(WINDOW / $spacing), 1), $n);
}

/** Pesos gaussianos de la ALMA para una ventana de k puntos. */
function almaWeights(int $k): array
{
    $m = OFFSET * ($k - 1);
```

```php
    $s = $k / SIGMA;
    $w = [];
    for ($j = 0; $j < $k; $j++) {
        $w[] = exp(-(($j - $m) ** 2) / (2 * $s ** 2));
    }

    return $w;
}

/** Perfil suavizado con n valores: se prolongan los extremos antes de filtrar. */
function smooth(array $elevation, int $k): array
{
    $before = intdiv($k - 1, 2);
    $after  = $k - 1 - $before;
    $padded = array_merge(
        array_fill(0, $before, $elevation[0]),
        $elevation,
        array_fill(0, $after, $elevation[count($elevation) - 1])
    );
    $w     = almaWeights($k);
    $total = array_sum($w);

    $result = [];
    for ($i = 0; $i < count($elevation); $i++) {
        $sum = 0.0;
        for ($j = 0; $j < $k; $j++) {
            $sum += $padded[$i + $j] * $w[$j];
        }
        $result[] = $sum / $total;
    }

    return $result;
}

/** Desnivel positivo y negativo acumulados de un perfil. */
function gainAndLoss(array $profile): array
{
    $gain = $loss = 0.0;
    for ($i = 1; $i < count($profile); $i++) {
        $diff = $profile[$i] - $profile[$i - 1];
        if ($diff >= 0) {
            $gain += $diff;
        } else {
            $loss -= $diff;
        }
    }

    return [$gain, $loss];
}

/**
 * Analiza las elevaciones de un track: lista de puntos [lat, lon, ele];
 * ele puede ser null. Devuelve el desnivel y las altitudes mínima y máxima.
 */
function analyze(array $points): array
{
    $elevation = array_values(array_map(
        'floatval',
```

```php
        array_filter(array_column($points, 2), static fn ($e) => $e !== null)
    )) ?: [0.0];
    $k = windowPoints($points, count($elevation));
    [$gain, $loss] = gainAndLoss(smooth($elevation, $k));
    if (distance($points[0], $points[count($points) - 1]) < CIRCUIT) {
        $loss = $gain;  // Circuito: D+ = D-
    }

    return ['k' => $k, 'gain' => $gain, 'loss' => $loss,
            'min' => min($elevation), 'max' => max($elevation)];
}

/** Ruido gaussiano reproducible: generador de Park y Miller y Box-Muller. */
final class Noise
{
    public function __construct(private int $state)
    {
    }

    public function uniform(): float
    {
        $this->state = $this->state * 48271 % 2147483647;

        return $this->state / 2147483647;
    }

    public function gauss(float $sigma): float
    {
        [$u1, $u2] = [$this->uniform(), $this->uniform()];

        return $sigma * sqrt(-2 * log($u1)) * cos(2 * M_PI * $u2);
    }
}

// Track sintético: subida de 500 m y bajada en 10 km, un punto cada 10 m,
// con ruido gaussiano de 3 m en la altitud
$noise = new Noise(1);
$n     = 1001;
$track = [];
for ($i = 0; $i < $n; $i++) {
    $x       = $i / ($n - 1);
    $ele     = 1000 + 500 * sin(M_PI * $x) + $noise->gauss(3);
    $track[] = [43.0 + 0.09 * $x, -2.0, $ele];
}

[$rawGain] = gainAndLoss(array_column($track, 2));
$result    = analyze($track);
printf("Desnivel sin suavizar: %.0f m\n", $rawGain);
printf("ALMA (k = %d): D+ %.0f m, D- %.0f m\n", $result['k'], $result['gain'],
    $result['loss']);
printf("Altitud: min %.0f m, max %.0f m\n", $result['min'], $result['max']);
```

D. Implementación en Rust

Implementación equivalente en Rust (edición 2021). Usa el mismo generador de ruido que el resto de implementaciones, por lo que debe producir la misma salida. Se compila y ejecuta con:

```
rustc -O alma_elevation.rs && ./alma_elevation
```

```rust
//! Análisis de elevaciones de un track GPS mediante la media móvil gaussiana (ALMA)
//! con una ventana de suavizado fijada en metros.

use std::f64::consts::PI;

const WINDOW: f64 = 100.0; // Anchura de la ventana de suavizado (m)
const OFFSET: f64 = 0.5; // Centro de la gaussiana dentro de la ventana (centrada)
const SIGMA: f64 = 6.0; // Factor de anchura de la gaussiana
const MIN_STEP: f64 = 1.0; // Separación mínima (m) para considerar que hay desplazamiento
const CIRCUIT: f64 = 100.0; // Distancia máxima (m) entre inicio y fin de un circuito
const R: f64 = 6371008.8; // Radio medio terrestre (m)

/// Un punto del track; `ele` es `None` si no hay altitud.
struct Point {
    lat: f64,
    lon: f64,
    ele: Option<f64>,
}

/// El resultado del análisis.
struct Analysis {
    k: usize,
    gain: f64,
    loss: f64,
    min: f64,
    max: f64,
}

/// Distancia (m) entre dos puntos por la fórmula del haversine.
fn distance(p: &Point, q: &Point) -> f64 {
    let (lat1, lat2) = (p.lat.to_radians(), q.lat.to_radians());
    let d_lat = lat2 - lat1;
    let d_lon = (q.lon - p.lon).to_radians();
    let h = (d_lat / 2.0).sin().powi(2) + lat1.cos() * lat2.cos() * (d_lon / 2.0).sin().powi(2);
    2.0 * R * h.sqrt().asin()
}

/// Número de puntos k de la ventana a partir de la separación media en movimiento.
fn window_points(points: &[Point], n: usize) -> usize {
    let steps: Vec<f64> = points
        .windows(2)
        .map(|w| distance(&w[0], &w[1]))
        .filter(|&d| d > MIN_STEP)
        .collect();
    if steps.is_empty() {
        return ((WINDOW / MIN_STEP) as usize).min(n);
    }
    let spacing = steps.iter().sum::<f64>() / steps.len() as f64;
    ((WINDOW / spacing).round() as usize).max(1).min(n)
}
```

```rust

/// Pesos gaussianos de la ALMA para una ventana de k puntos.
fn alma_weights(k: usize) -> Vec<f64> {
    let m = OFFSET * (k as f64 - 1.0);
    let s = k as f64 / SIGMA;
    (0..k)
        .map(|j| (-(j as f64 - m).powi(2) / (2.0 * s * s)).exp())
        .collect()
}

/// Perfil suavizado con n valores: se prolongan los extremos antes de filtrar.
fn smooth(elevation: &[f64], k: usize) -> Vec<f64> {
    let before = (k - 1) / 2;
    let after = k - 1 - before;
    let first = elevation[0];
    let last = elevation[elevation.len() - 1];
    let padded: Vec<f64> = std::iter::repeat(first)
        .take(before)
        .chain(elevation.iter().copied())
        .chain(std::iter::repeat(last).take(after))
        .collect();
    let w = alma_weights(k);
    let total: f64 = w.iter().sum();
    padded
        .windows(k)
        .map(|win| win.iter().zip(&w).map(|(x, wj)| x * wj).sum::<f64>() / total)
        .collect()
}

/// Desnivel positivo y negativo acumulados de un perfil.
fn gain_and_loss(profile: &[f64]) -> (f64, f64) {
    profile.windows(2).fold((0.0, 0.0), |(gain, loss), w| {
        let diff = w[1] - w[0];
        if diff >= 0.0 {
            (gain + diff, loss)
        } else {
            (gain, loss - diff)
        }
    })
}

/// Analiza las elevaciones de un track.
fn analyze(points: &[Point]) -> Analysis {
    let mut elevation: Vec<f64> = points.iter().filter_map(|p| p.ele).collect();
    if elevation.is_empty() {
        elevation.push(0.0);
    }
    let k = window_points(points, elevation.len());
    let (gain, mut loss) = gain_and_loss(&smooth(&elevation, k));
    if distance(&points[0], &points[points.len() - 1]) < CIRCUIT {
        loss = gain; // Circuito: D+ = D-
    }
    let min = elevation.iter().copied().fold(f64::INFINITY, f64::min);
    let max = elevation.iter().copied().fold(f64::NEG_INFINITY, f64::max);
    Analysis { k, gain, loss, min, max }
}

/// Ruido gaussiano reproducible: generador de Park y Miller y Box-Muller.
```

```rust
struct Noise {
    state: u64,
}

impl Noise {
    fn uniform(&mut self) -> f64 {
        self.state = self.state * 48271 % 2147483647;
        self.state as f64 / 2147483647.0
    }

    fn gauss(&mut self, sigma: f64) -> f64 {
        let (u1, u2) = (self.uniform(), self.uniform());
        sigma * (-2.0 * u1.ln()).sqrt() * (2.0 * PI * u2).cos()
    }
}

fn main() {
    // Track sintético: subida de 500 m y bajada en 10 km, un punto cada 10 m,
    // con ruido gaussiano de 3 m en la altitud
    let mut noise = Noise { state: 1 };
    let n = 1001;
    let track: Vec<Point> = (0..n)
        .map(|i| {
            let x = i as f64 / (n - 1) as f64;
            let ele = 1000.0 + 500.0 * (PI * x).sin() + noise.gauss(3.0);
            Point { lat: 43.0 + 0.09 * x, lon: -2.0, ele: Some(ele) }
        })
        .collect();

    let raw: Vec<f64> = track.iter().filter_map(|p| p.ele).collect();
    let result = analyze(&track);
    println!("Desnivel sin suavizar: {:.0} m", gain_and_loss(&raw).0);
    println!(
        "ALMA (k = {}): D+ {:.0} m, D- {:.0} m",
        result.k, result.gain, result.loss
    );
    println!("Altitud: min {:.0} m, max {:.0} m", result.min, result.max);
}
```